

NLP Text Preprocessing & Sentence Classification

Technical Documentation & Execution Notes

Overview

This project demonstrates an end-to-end Classical Natural Language Processing (NLP) pipeline built in Python. The workflow covers string sanitization, tokenization, stop-word removal, stemming, lemmatization, Bag-of-Words vectorization, and model training using a Naive Bayes classifier to perform binary sentiment analysis on product feedback.

Environment & Toolkit

Package / Tool	Purpose	Role in Project
Pandas & NumPy	Data Structuring	Managing text corpora and label mappings in tabular format
NLTK	NLP Preprocessing	Tokenization, stop-word filtering, Porter stemming, and WordNet lemmatization
Scikit-Learn	Feature Extraction & Modeling	<code>CountVectorizer</code> for Bag-of-Words and <code>MultinomialNB</code> for sentiment classification

Implementation Script

```
import pandas as pd
import numpy as np
import re
import string
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Download NLTK resources
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
try:
    nltk.download('punkt_tab')
except:
```

```

pass

# Sample dataset
data = {
    "Text": [
        "I love this phone", "This mobile is excellent", "Amazing camera quality",
        "Very good battery", "I like this product", "I hate this phone",
        "Worst mobile ever", "Very bad battery", "Poor camera quality", "I dislike this
product"
    ],
    "Sentiment": [
        "Positive", "Positive", "Positive", "Positive", "Positive",
        "Negative", "Negative", "Negative", "Negative", "Negative"
    ]
}

df = pd.DataFrame(data)

# Text Preprocessing Steps
df["Text"] = df["Text"].str.lower()
df["Text"] = df["Text"].str.replace('[{}]' .format(string.punctuation), '', regex=True)
df["Text"] = df["Text"].str.replace(r'\d+', '', regex=True)
df["Text"] = df["Text"].apply(word_tokenize)

stop_words = set(stopwords.words('english'))
df["Text"] = df["Text"].apply(lambda words: [word for word in words if word not in
stop_words])

stemmer = PorterStemmer()
df["Text"] = df["Text"].apply(lambda words: [stemmer.stem(word) for word in words])

lemmatizer = WordNetLemmatizer()
df["Text"] = df["Text"].apply(lambda words: [lemmatizer.lemmatize(word) for word in
words])
df["Text"] = df["Text"].apply(lambda words: " ".join(words))

# Vectorization & Model Training
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df["Text"])
y = df["Sentiment"]

model = MultinomialNB()
model.fit(X, y)

# Single Sentence Inference
sentence = "This phone has good camera"
sentence = sentence.lower()
sentence = re.sub(r'[\w\s]', '', sentence)
sentence = re.sub(r'\d+', '', sentence)
words = word_tokenize(sentence)
words = [word for word in words if word not in stop_words]
words = [stemmer.stem(word) for word in words]
words = [lemmatizer.lemmatize(word) for word in words]
sentence = " ".join(words)

test = vectorizer.transform([sentence])

```

```
prediction = model.predict(test)
print("Test Sentence:", sentence)
print("Predicted Sentiment:", prediction[0])
```

Preprocessing & Execution Steps

- **Sanitization:** Case normalization via `lower()`, combined with regex filtering to strip punctuation and digits.
- **Tokenization & Filtering:** Converts character sequences into lists of tokens using `word_tokenize` and removes English stop words.
- **Stemming & Lemmatization:** Reduces word variations to base roots (e.g., using `PorterStemmer` and `WordNetLemmatizer`).
- **Vectorization:** Transforms processed text into term-frequency sparse vectors via `CountVectorizer`.
- **Classification:** Fits a `MultinomialNB` classifier on the sparse feature matrix and evaluates single-sentence inference.

Technical Recommendations

- **Pipeline Order:** Applying stemming directly before lemmatization can sometimes truncate words into non-standard forms; applying lemmatization prior to stemming generally yields better root quality.
- **Corpus Expansion:** To prevent vocabulary mismatch during production inference, expand training samples beyond the initial 10-row toy dataset.